

One Data, Many Models: Fixed, Mixed, Smooth, and Correlated Regression in R

Ikedichi Edward Azuh

4/8/2021

Introduction

Regression analysis is one of the most widely used statistical tools in ecology, biology, environmental science, and many other disciplines. Although the simple linear model is often the starting point, real data frequently contain additional complexities such as grouped observations, hierarchical structure, and correlation among observations. Consequently, several modeling frameworks have been developed to address these challenges while still operating under the assumption of a Gaussian (normal) response variable.

This lecture introduces and compares several approaches for modeling Gaussian data in R, including classical linear models (`lm()`), linear mixed-effects models (`lme()` and `lmer()`), generalized additive models (`gam()`), generalized linear mixed models fitted with `glmmTMB()`, and generalized least squares models (`gls()`). Using a simulated dataset with group-specific intercepts and slopes, we demonstrate how each method represents the same underlying relationship between the response and explanatory variable, while differing in their assumptions about group-level variation and residual structure.

By the end of this lecture, students should be able to:

- Understand the differences between fixed-effects, mixed-effects, smooth, and correlated-error models.
- Recognize when grouped or hierarchical data require more than a simple linear regression.
- Fit and interpret Gaussian regression models using several commonly used R packages.
- Perform appropriate model diagnostics for different model classes.
- Compare competing models using information criteria and predictive performance metrics.
- Appreciate that different statistical frameworks can often yield similar conclusions while providing different interpretations of variation in the data.

The overarching goal of this lecture is to develop an intuitive understanding of how different regression frameworks relate to one another and to provide practical guidance on selecting and interpreting models for Gaussian response data in R.

1.0 The data simulation

We simulate a hierarchical data with group-specific intercepts and slopes

```
N <- seq(50, by=25, len=8)
set.seed(1)
x <- seq(0, 100, len=1100)
```

```

intercept <- rnorm(8, 10, 3)
slope <- rnorm(8, 4, 1)

make.data <- function(j){

  x <- sample(x, size = N[j], replace = FALSE)

  err <- rnorm(N[j], 0, 23)

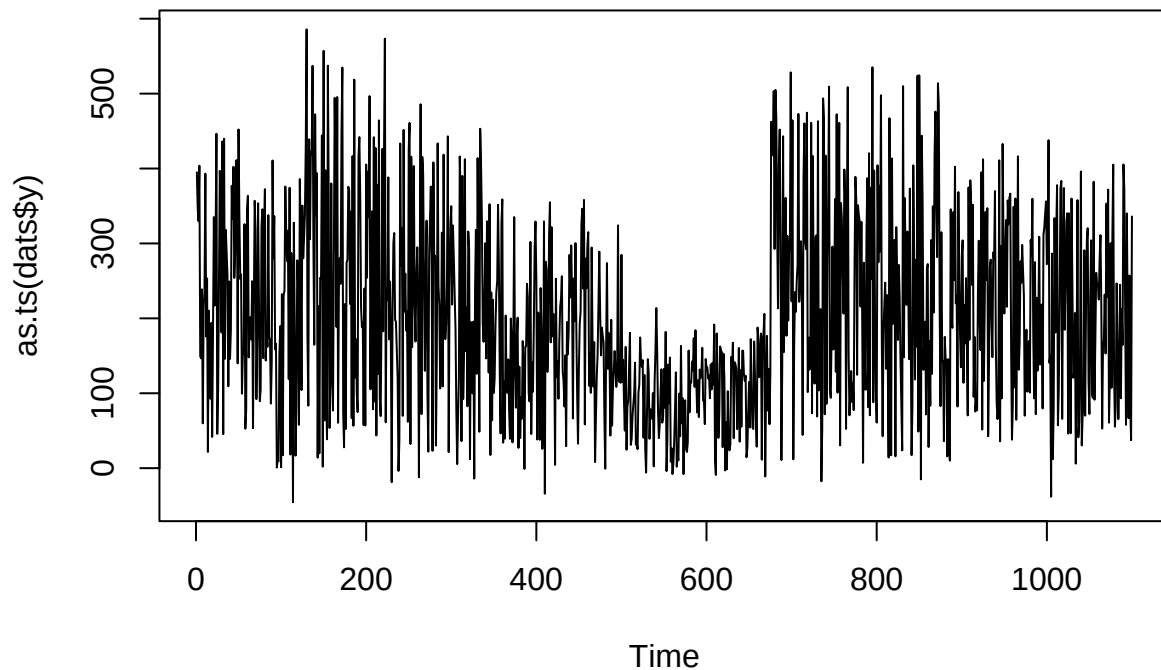
  y <- intercept[j] + slope[j] * x + err

  data.frame(
    y = y,
    x = x,
    group = factor(j)
  )
}

dats <- do.call(rbind, lapply(1:8, make.data))
#dats <- make.data(1)
#for (i in 2:length(N)) dats <- rbind.data.frame(dats, make.data(i))

dats$group <- as.factor(dats$group)
plot(as.ts(dats$y))

```

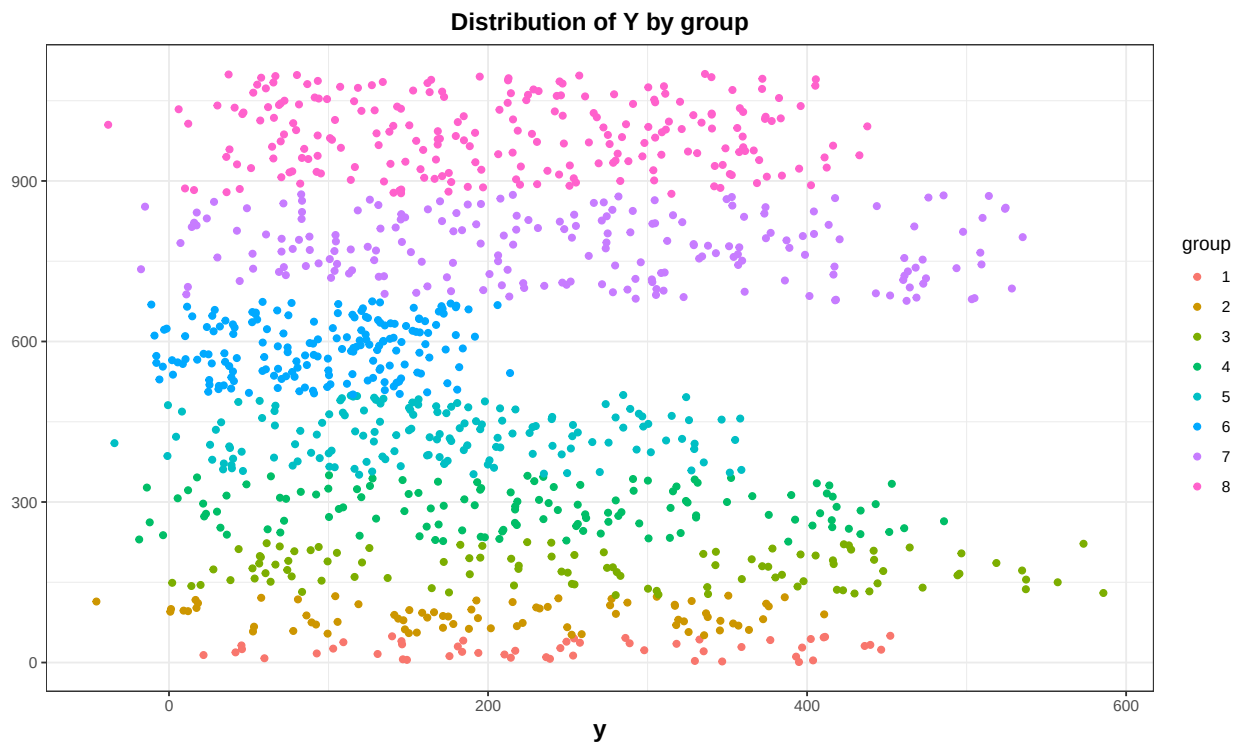


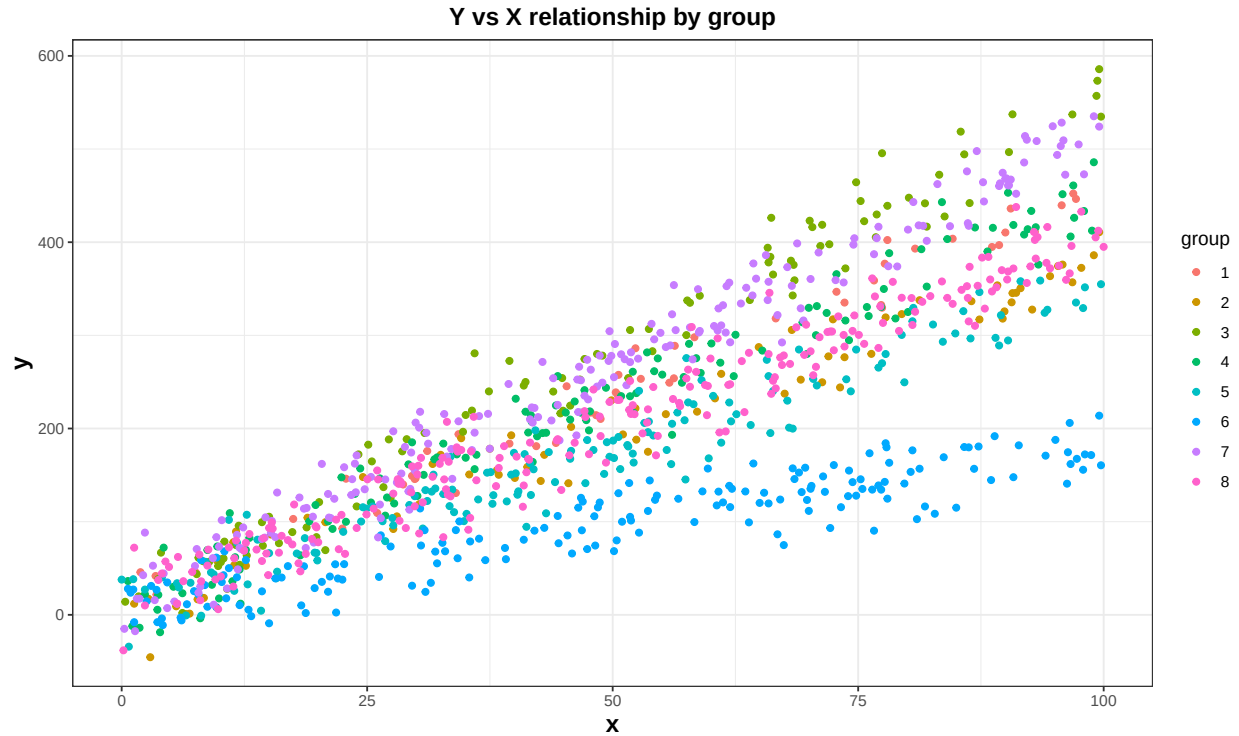
2.0 Data visualization plot

```
library(ggplot2)

a <- ggplot(dats, aes(y=1:nrow(dats),x=y, color=group))+geom_point() +
  ggtitle("Distribution of Y by group") +
  theme_bw() +
  theme(plot.title=element_text(color="black",size=14,face="bold",hjust=0.5),
        axis.title.x = element_text(color="black", size=14, face="bold"),
        axis.title.y = element_blank()
  )

b <- ggplot(dats, aes(x,y, color=group))+geom_point() +
  ggtitle("Y vs X relationship by group") +
  theme_bw() +
  theme(plot.title=element_text(color="black",size=14,face="bold",hjust=0.5),
        axis.title.x = element_text(color="black", size=14, face="bold"),
        axis.title.y = element_text(color="black", size=14, face="bold")
  )
par(mfrow=c(1,2))
a;b
```





3.0 Models fitting

3.1 Fixed effect model using `lm()`

Fixed-effect model allowing group-specific intercepts and slopes (no pooling).

```
library(nlme)
library(lme4)
library(mgcv)
library(glmmTMB)

summary(fit.lm <- lm(y ~ x*group, dats))

##
## Call:
## lm(formula = y ~ x * group, data = dats)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -67.435 -14.912   0.118  16.676  74.825
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  16.751032   7.352225   2.278 0.022899 *
## x             4.431110   0.123593  35.853 < 2e-16 ***
## group2       -5.682104   9.144446  -0.621 0.534485
```

```
## group3      -9.982071    8.677110   -1.150  0.250237
## group4      -6.564639    8.474478   -0.775  0.438723
## group5      -8.034848    8.383296   -0.958  0.338057
## group6      -7.024693    8.176599   -0.859  0.390463
## group7      -4.819375    8.075959   -0.597  0.550795
## group8      -3.091920    8.040274   -0.385  0.700643
## x:group2     -0.700617    0.153120   -4.576  5.30e-06 ***
## x:group3      1.109225    0.149207    7.434  2.13e-13 ***
## x:group4     -0.005737    0.144578   -0.040  0.968352
## x:group5     -0.996210    0.144845   -6.878  1.03e-11 ***
## x:group6     -2.683747    0.138532  -19.373 < 2e-16 ***
## x:group7      0.655473    0.137583    4.764  2.15e-06 ***
## x:group8     -0.510561    0.136053   -3.753  0.000184 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 23.98 on 1084 degrees of freedom
## Multiple R-squared:  0.9665, Adjusted R-squared:  0.966
## F-statistic: 2086 on 15 and 1084 DF, p-value: < 2.2e-16
```

3.2 Mixed effects models using lmer(), lm() and glmmTMB()

Slope and intercept are random.

```
summary(fit.lmer <- lmer(y ~ x + (x|group),dats))
```

```
## Linear mixed model fit by REML ['lmerMod']
## Formula: y ~ x + (x | group)
## Data: dats
##
## REML criterion at convergence: 10157.7
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -2.81108 -0.64352 -0.00151  0.67985  3.06476
##
## Random effects:
## Groups   Name                Variance Std.Dev. Corr
## group    (Intercept)          0.000    0.000
##          x                   1.344    1.159    NaN
## Residual                    572.955  23.936
## Number of obs: 1100, groups: group, 8
##
## Fixed effects:
##              Estimate Std. Error t value
## (Intercept)  10.9131    1.4413    7.571
## x             4.0416    0.4108    9.839
##
## Correlation of Fixed Effects:
##  (Intr)
## x -0.054
## optimizer (nloptwrap) convergence code: 0 (OK)
## boundary (singular) fit: see help('isSingular')
```

```
summary(fit.tmb <- glmmTMB(y ~ x + (x|group), dats))
```

```
## Family: gaussian ( identity )
## Formula:          y ~ x + (x | group)
## Data: dats
##
##      AIC      BIC    logLik -2*log(L)  df.resid
## 10172.2 10202.2 -5080.1  10160.2    1094
##
## Random effects:
##
## Conditional model:
##   Groups   Name      Variance Std.Dev. Corr
##   group   (Intercept)  0.04844  0.2201
##         x             1.16790  1.0807  1.00
##   Residual              572.42075 23.9253
## Number of obs: 1100, groups: group, 8
##
## Dispersion estimate for gaussian family (sigma^2): 572
##
## Conditional model:
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept)   10.933      1.448    7.552 4.29e-14 ***
## x              4.041      0.383   10.552 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(fit.lme <- lme(y ~ x , random = ~ x|group, dats))
```

```
## Linear mixed-effects model fit by REML
##   Data: dats
##      AIC      BIC    logLik
## 10169.65 10199.66 -5078.826
##
## Random effects:
##   Formula: ~x | group
##   Structure: General positive-definite, Log-Cholesky parametrization
##              StdDev    Corr
## (Intercept)  0.1150536 (Intr)
## x            1.1585368 0.274
## Residual     23.9364343
##
## Fixed effects: y ~ x
##              Value Std.Error   DF  t-value p-value
## (Intercept) 10.915528 1.4420138 1091  7.569642      0
## x           4.041594 0.4104396 1091  9.846989      0
##   Correlation:
##   (Intr)
## x -0.046
##
## Standardized Within-Group Residuals:
##              Min           Q1           Med           Q3           Max
```

```
## -2.810841901 -0.643584867 -0.002182158 0.679697338 3.064748539
##
## Number of Observations: 1100
## Number of Groups: 8
```

3.3 Mixed effects (smooth) model using gam()

`s(group, x, bs = "re")`: this formulation represents group-specific deviations in the x-effect using penalized regression, in a way that is closely related to a random slope model.

```
summary(fit.gam <- gam(y ~ x + s(group,x,bs = 're'),data = dats, method = 'REML'))
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## y ~ x + s(group, x, bs = "re")
##
## Parametric coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 10.9131    1.4414    7.571 7.84e-14 ***
## x           4.0416     0.4105    9.846 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##             edf Ref.df   F p-value
## s(group,x) 6.992     7 1151 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) = 0.966   Deviance explained = 96.6%
## -REML = 5078.8   Scale est. = 572.96    n = 1100
```

3.4 Correlated error model using gls()

Although the data do not contain true autocorrelation, GLS is included to demonstrate how a model with an explicit correlation structure behaves under a correctly specified vs misspecified setting.

```
summary(fit.gls <- gls(y ~ x*group , corr = corARMA(form = ~1|group,p=2,q=0), data=dats))
```

```
## Generalized least squares fit by REML
## Model: y ~ x * group
## Data: dats
##      AIC      BIC    logLik
## 10130.07 10224.85 -5046.037
##
## Correlation Structure: ARMA(2,0)
## Formula: ~1 | group
## Parameter estimate(s):
```

```

##          Phi1          Phi2
## -0.054903806  0.006217807
##
## Coefficients:
##              Value Std.Error   t-value p-value
## (Intercept) 16.721757  7.264164    2.30195  0.0215
## x            4.431824  0.123447   35.90075  0.0000
## group2      -5.531401  9.017738   -0.61339  0.5397
## group3      -9.977640  8.584776   -1.16225  0.2454
## group4      -6.463842  8.374556   -0.77184  0.4404
## group5      -7.865737  8.267242   -0.95143  0.3416
## group6      -7.195649  8.066687   -0.89202  0.3726
## group7      -4.888927  7.980298   -0.61262  0.5403
## group8      -2.851803  7.940929   -0.35913  0.7196
## x:group2     -0.703483  0.152688   -4.60734  0.0000
## x:group3      1.109327  0.149435    7.42349  0.0000
## x:group4     -0.008200  0.144505   -0.05675  0.9548
## x:group5     -0.999901  0.144270   -6.93075  0.0000
## x:group6     -2.680473  0.138122  -19.40661  0.0000
## x:group7      0.656715  0.137486    4.77660  0.0000
## x:group8     -0.515343  0.135839   -3.79379  0.0002
##
## Correlation:
##      (Intr) x      group2 group3 group4 group5 group6 group7 group8 x:grp2
## x      -0.896
## group2 -0.806  0.721
## group3 -0.846  0.758  0.682
## group4 -0.867  0.777  0.699  0.734
## group5 -0.879  0.787  0.708  0.744  0.762
## group6 -0.901  0.807  0.725  0.762  0.781  0.791
## group7 -0.910  0.815  0.733  0.770  0.790  0.800  0.820
## group8 -0.915  0.819  0.737  0.774  0.793  0.804  0.824  0.833
## x:group2 0.724 -0.808 -0.887 -0.613 -0.628 -0.636 -0.652 -0.659 -0.662
## x:group3 0.740 -0.826 -0.596 -0.886 -0.642 -0.650 -0.666 -0.673 -0.677  0.668
## x:group4 0.765 -0.854 -0.616 -0.647 -0.889 -0.672 -0.689 -0.696 -0.700  0.691
## x:group5 0.766 -0.856 -0.617 -0.648 -0.665 -0.891 -0.690 -0.698 -0.701  0.692
## x:group6 0.800 -0.894 -0.645 -0.677 -0.694 -0.703 -0.891 -0.729 -0.732  0.723
## x:group7 0.804 -0.898 -0.648 -0.680 -0.698 -0.707 -0.724 -0.891 -0.736  0.726
## x:group8 0.814 -0.909 -0.656 -0.689 -0.706 -0.715 -0.733 -0.741 -0.893  0.735
##      x:grp3 x:grp4 x:grp5 x:grp6 x:grp7
## x
## group2
## group3
## group4
## group5
## group6
## group7
## group8
## x:group2
## x:group3
## x:group4 0.706
## x:group5 0.707 0.731
## x:group6 0.738 0.764 0.765
## x:group7 0.742 0.767 0.768 0.802

```



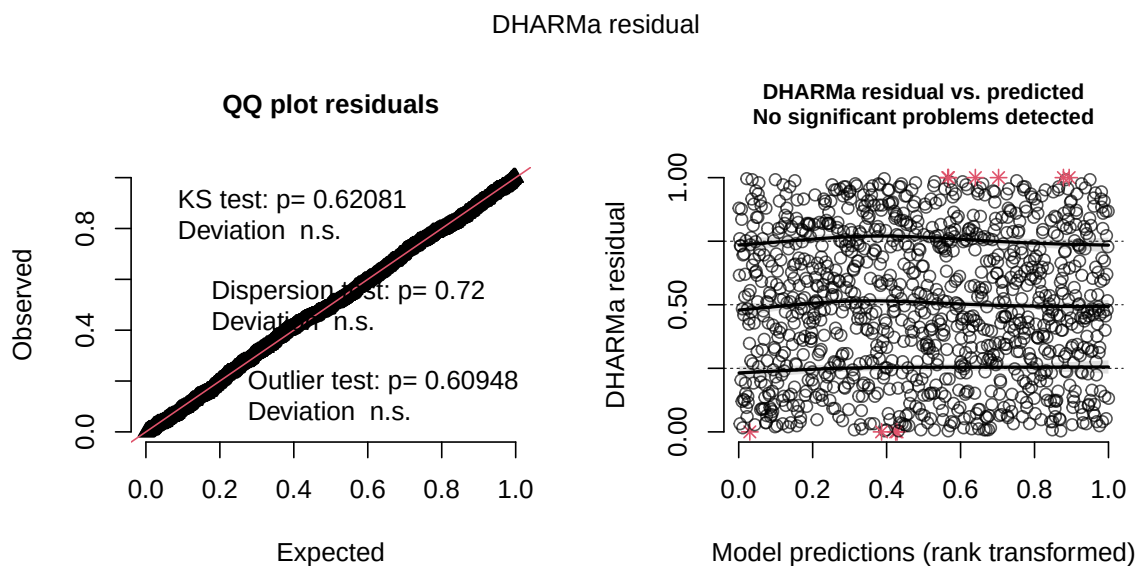
```
## x:group8 0.751 0.776 0.778 0.812 0.816
##
## Standardized residuals:
##      Min      Q1      Med      Q3      Max
## -2.81748222 -0.62386691 0.00304544 0.69479778 3.12073507
##
## Residual standard error: 23.97542
## Degrees of freedom: 1100 total; 1084 residual
```

4.0 Model Diagnostics

4.1 lm

```
library(DHARMa)

simulationOutput <- simulateResiduals(fittedModel = fit.lm, plot = T)
```

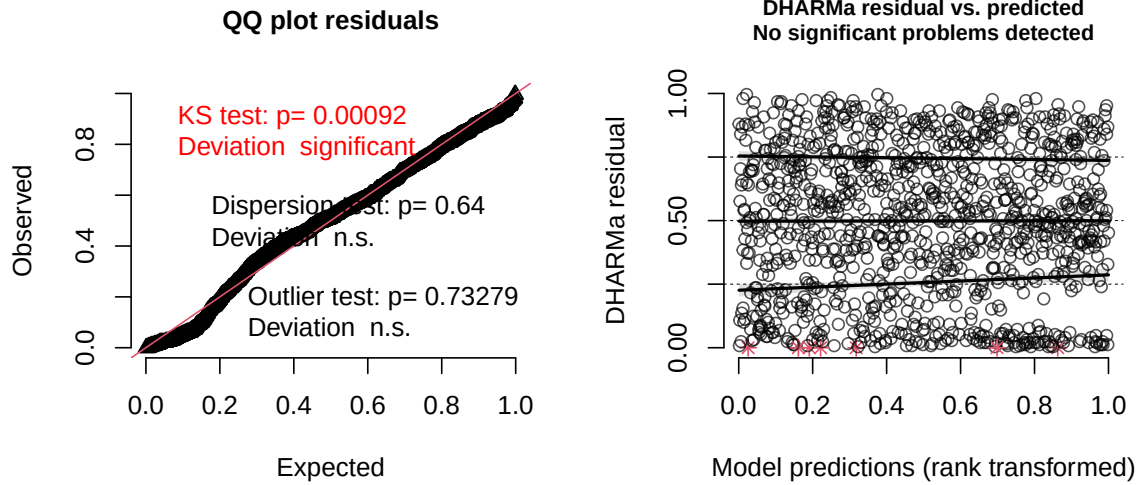


```
#acf(residuals(fit.lm)) no sign of autocorrelation
```

4.2 lmer

```
simulationOutput <- simulateResiduals(fittedModel = fit.lmer, plot = T)
```

DHARMA residual

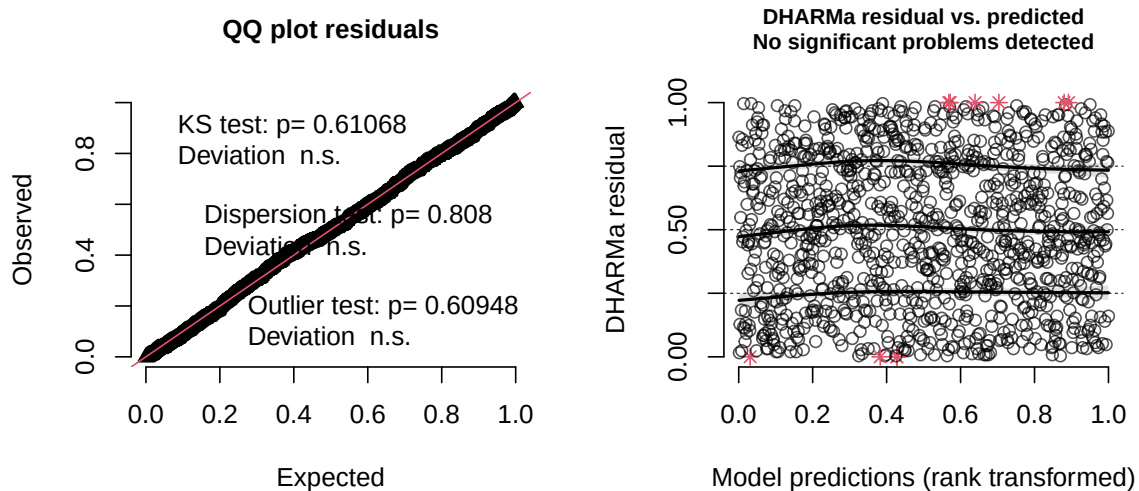


```
#acf(residuals(fit.lmer))
```

4.3 gam

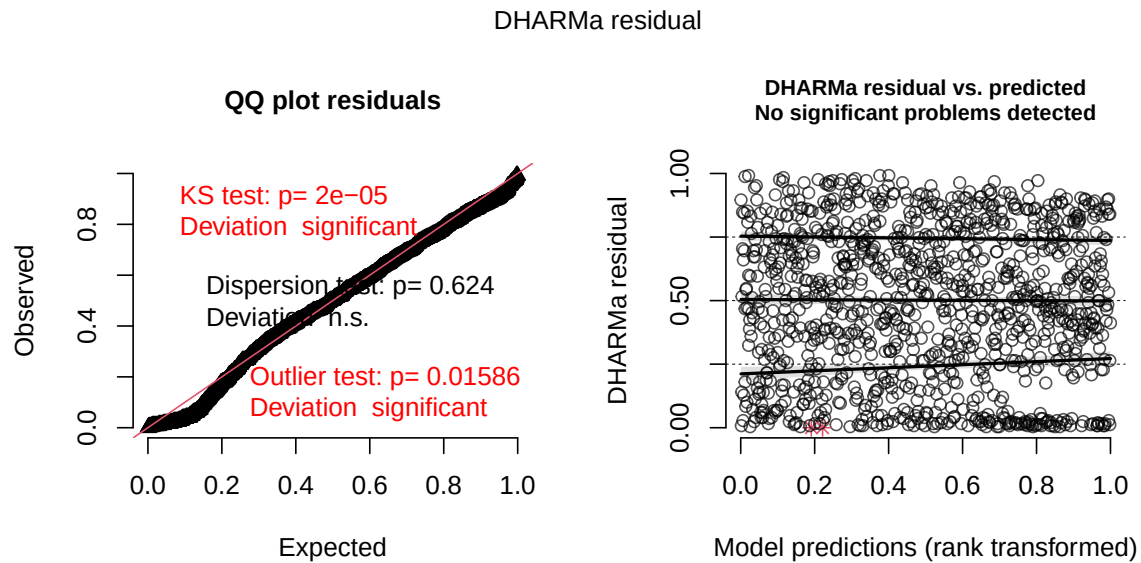
```
#install.packages("mgcViz")
simulationOutput <- simulateResiduals(fittedModel = fit.gam, plot = T)
```

DHARMA residual



4.4 TMB

```
simulationOutput <- simulateResiduals(fittedModel = fit.tmb, plot = T)
```



4.5 lme

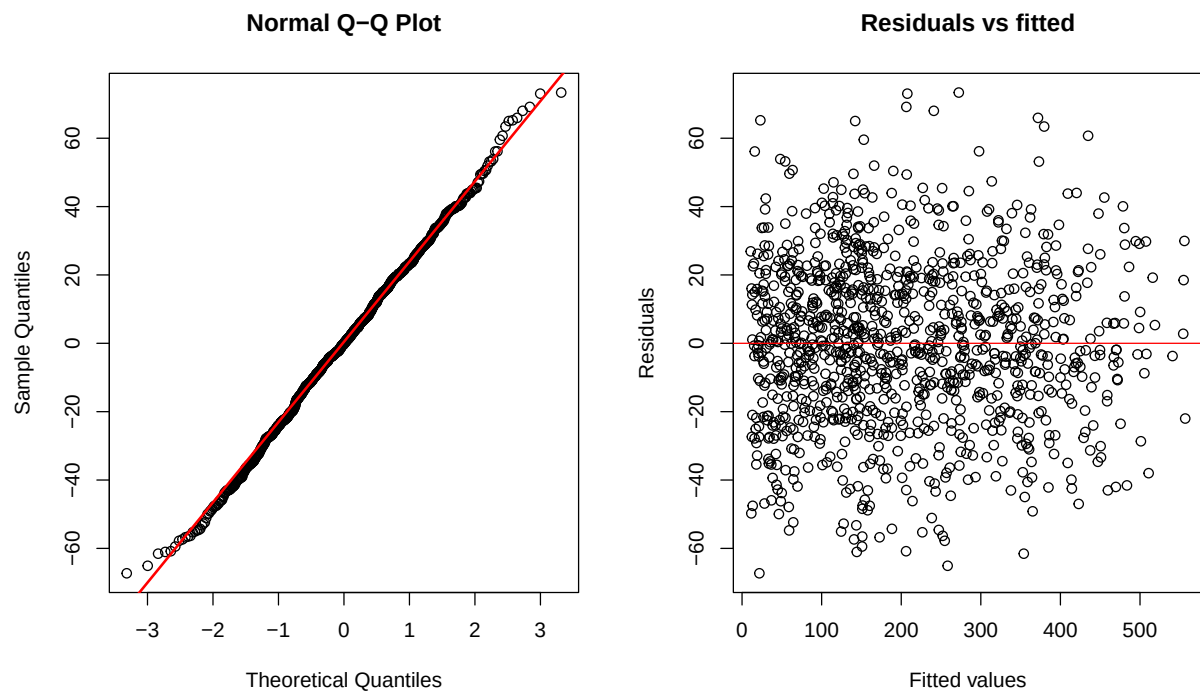
DHARMA package is not directly useable for lme and gls. So we use classic approach.

```
par(mfrow = c(1,2))

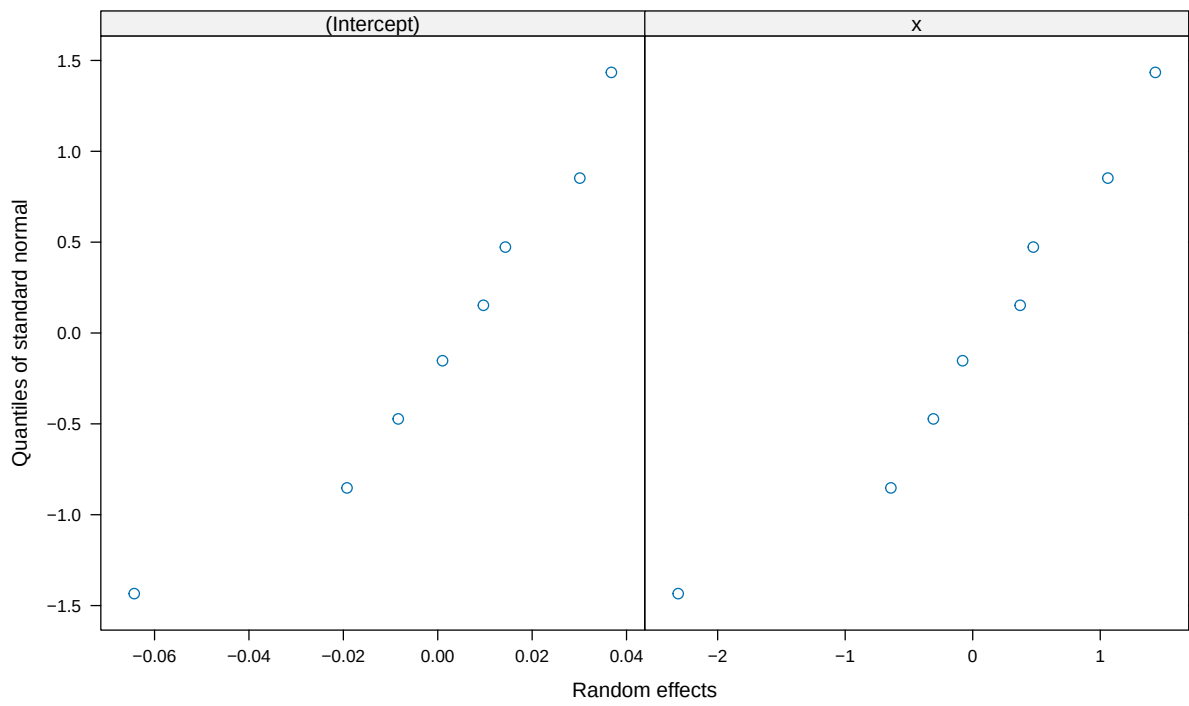
res <- residuals(fit.lme)

qqnorm(res)
qqline(res, col = "red", lwd = 2)

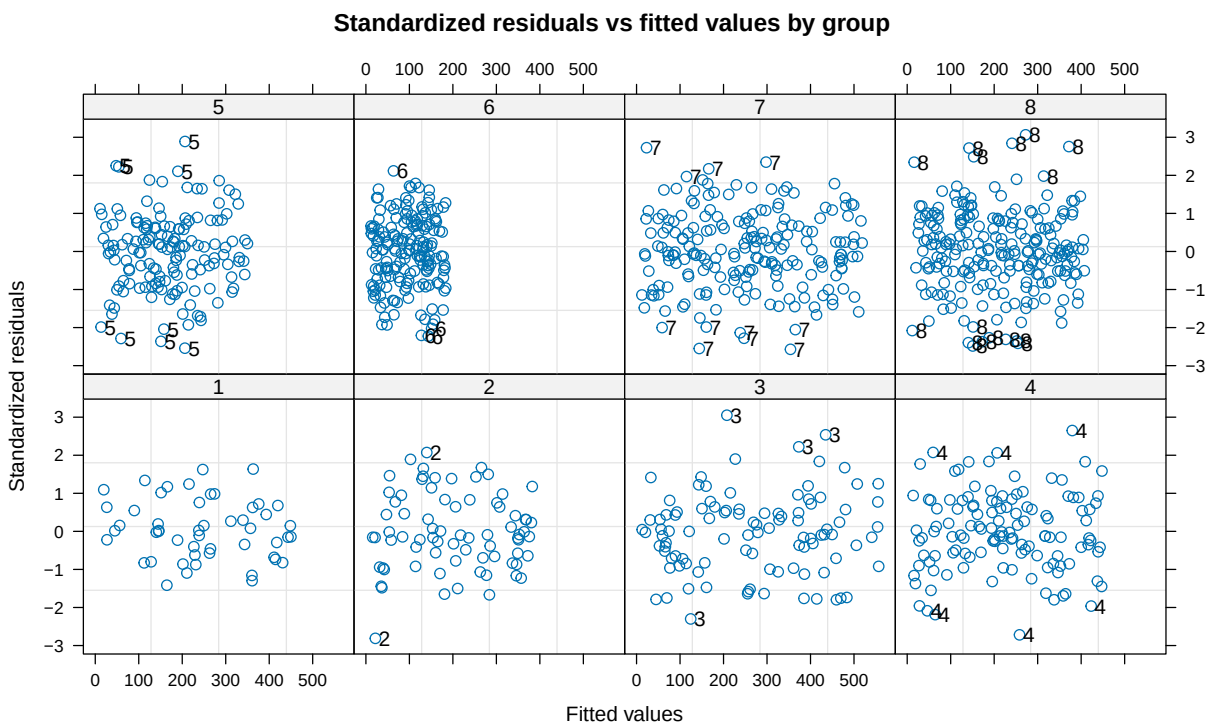
plot(fitted(fit.lme), res,
     xlab = "Fitted values",
     ylab = "Residuals",
     main = "Residuals vs fitted")
abline(h = 0, col = "red")
```



```
par(mfrow = c(1,1))
qqnorm(fit.lme, ~ ranef(., level=1))
```



```
plot(fit.lme, resid(., type = "p") ~ fitted(.) | group,
     id = 0.05, adj = -0.3, main="Standardized residuals vs fitted values by group") # to check for eq
```



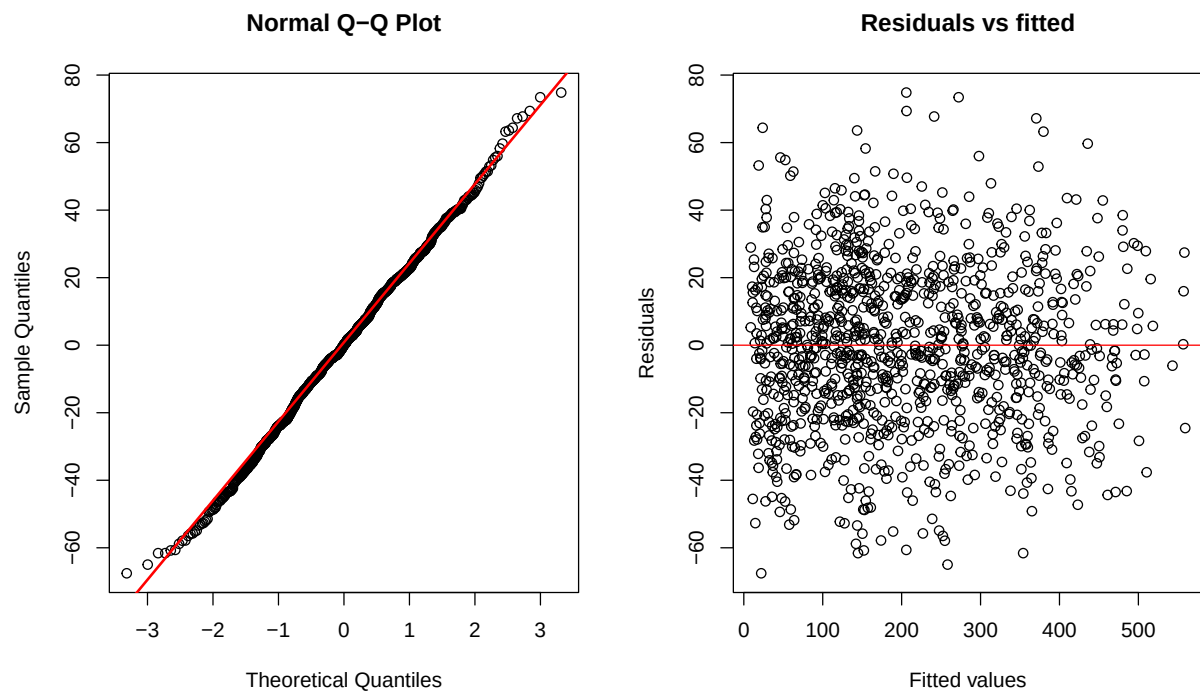
4.6 gls

```
par(mfrow = c(1,2))

res <- residuals(fit.gls)

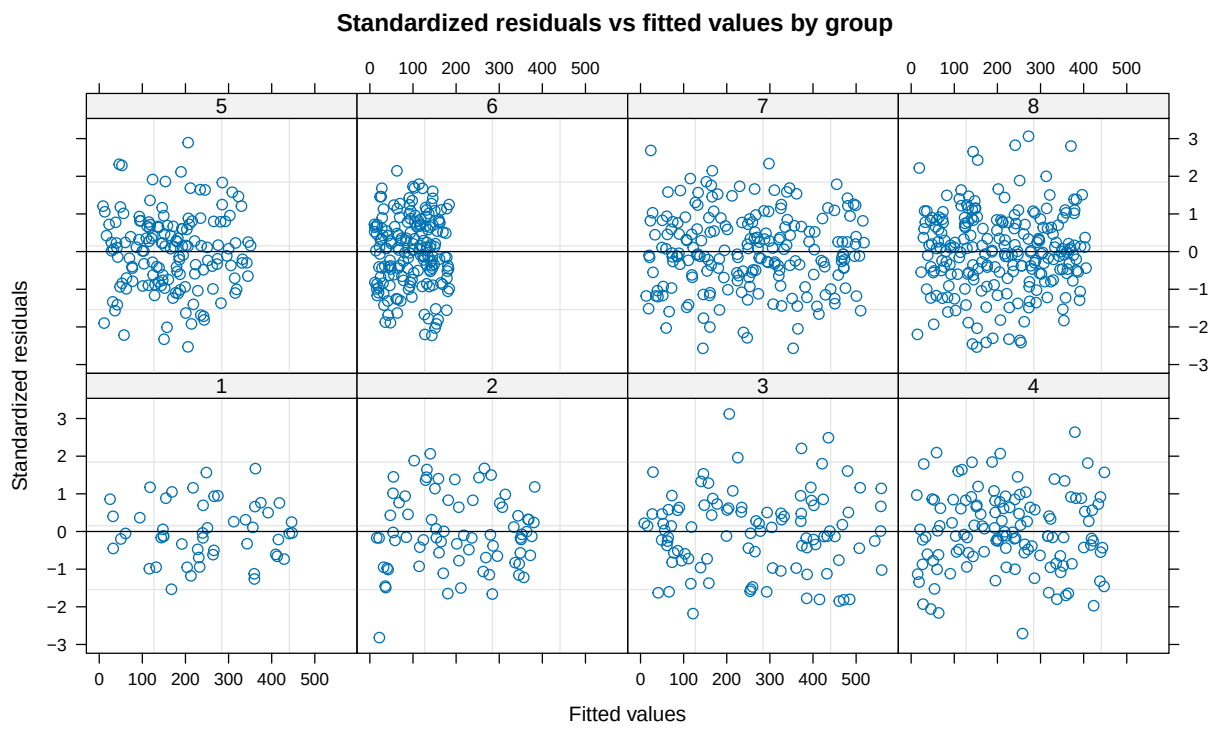
qqnorm(res)
qqline(res, col = "red", lwd = 2)

plot(fitted(fit.gls), res,
     xlab = "Fitted values",
     ylab = "Residuals",
     main = "Residuals vs fitted")
abline(h = 0, col = "red")
```

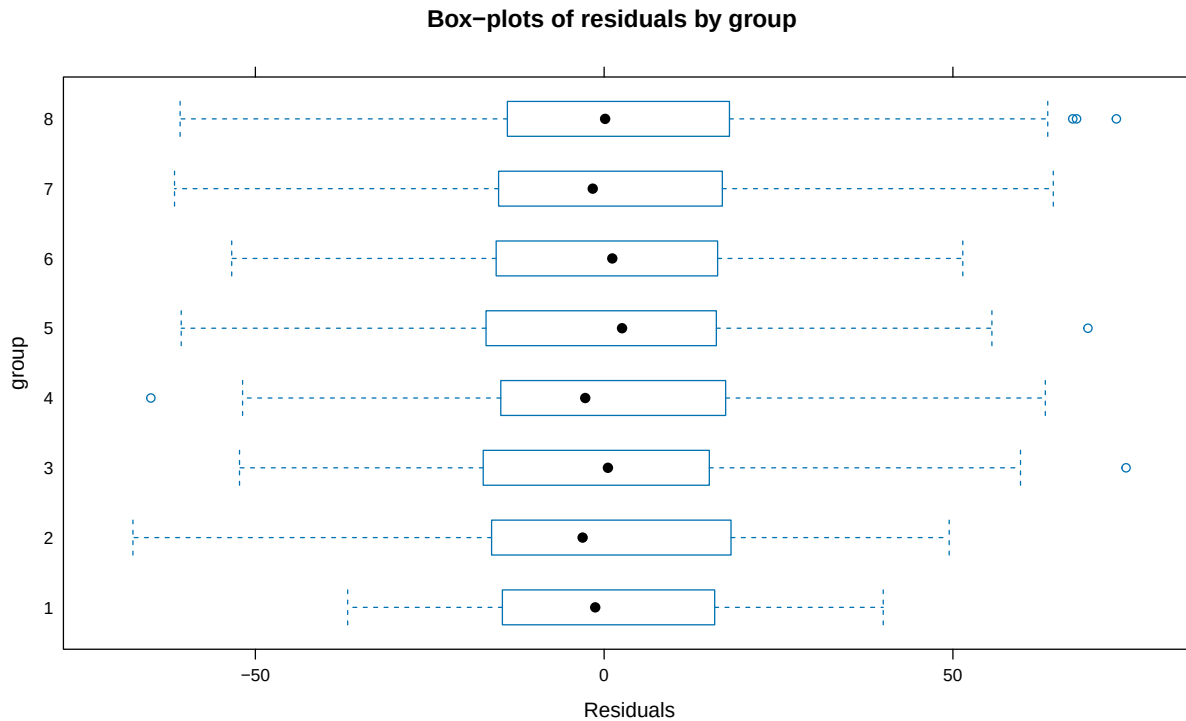


```
par(mfrow = c(1,1))

plot(fit.gls, resid(., type = "p") ~ fitted(.) | group, abline = 0, main="Standardized residuals vs fitted values by group")
```



```
plot(fit.gls, group ~ resid(.), main="Box-plots of residuals by group")
```



5.0 Model Comparison

All models yield similar predictive performance under gaussian assumptions, but differ in how they allocate variation between group effects and residual error.

```
library(Metrics)
library(knitr)

get_metrics <- function(model, name){

  pred <- predict(model, newdata = dats, allow.new.levels = TRUE)

  data.frame(
    Model = name,
    AIC = AIC(model),
    RMSE = rmse(dats$y, pred),
    MAE = mae(dats$y, pred)
  )
}

results <- rbind(
  get_metrics(fit.lm, "lm"),
  get_metrics(fit.lmer, "lmer"),
  get_metrics(fit.gam, "gam"),
  get_metrics(fit.tmb, "tmb"),
```

```

  get_metrics(fit.lme, "lme"),
  get_metrics(fit.gls, "glS")
)

knitr::kable(results, caption = "Model comparison")

```

Table 1: Model comparison

Model	AIC	RMSE	MAE
lm	10129.80	23.80899	18.94142
lmer	10169.66	23.83850	18.93616
gam	10118.52	23.83850	18.93616
tmb	10172.18	23.83822	18.93913
lme	10169.65	23.83838	18.93649
glS	10130.07	23.80913	18.93937